

Interacting with HTML from JavaScript

Selectors, event listeners, dynamic rendering, and CSS class manipulation.

The Document Object Model

Live Tree Structure

Browser converts HTML strings into programmable node trees.

The global `document`

JavaScript entry point for page traversal.

Two-Way Sync

Modifying the JS object immediately updates visual rendering.



Finding Elements



.getElementById()

Extremely fast ID lookup.



.querySelector()

Returns first matching CSS selector.



.querySelectorAll()

Returns NodeList of all matches.

Targeting Assignment Elements

INDEX.HTML

```
<nav>
  <button id="btn-all">All</button>
  <button id="btn-
laptops">Laptops</button>
</nav>

<!-- Empty grid for injection -->
<div id="product-container"></div>
```



```
// Grab buttons for events
const btnAll = document.getElementById('btn-all');
const btnLap = document.getElementById('btn-laptops');

// Grab container for rendering
const container = document.getElementById('product-
container');
```

Injecting Data

.textContent

Safely updates raw text only.

```
const title = document.querySelector('h1');  
  
// Treats tags as literal strings  
title.textContent = "<strong>Store</strong>";
```

.innerHTML

Parses string tags into actual DOM nodes.

```
const div = document.getElementById('box');  
  
// Renders actual bold text  
div.innerHTML = "<strong>Sale!</strong>";
```

Array to HTML String Mapping

Combine template literals with `map()` and `join("")` to efficiently generate complex markup from arrays.

```
function renderProducts(items) {  
  const htmlString = items.map(item => `  
    <div class="product-card">  
      <h3>${item.name}</h3>  
      <p>${item.price}</p>  
    </div>  
  `).join(""); // Removes array commas  
  
  container.innerHTML = htmlString;  
}
```

Modifying CSS Classes



.classList.add('x')
Appends CSS class.



.classList.remove('x')
Removes CSS class.

TERNARY CLASS INJECTION

```
const itemsHTML = data.map(item => `  <h3>${item.name}</h3>  
</div>  
`);
```

Injecting logic directly into templates.

Listening for User Actions

Target Element

The DOM node receiving the interaction.

Event Type

String identifier: 'click', 'input', 'submit'.

Callback Handler

Function executed when event occurs.

```
button.addEventListener('click', () => {  
  console.log('Button clicked!');  
});
```

The Filter & Render Workflow

- 1 Detect 'click' on category button.
- 2 Filter source array using `.filter()`.
- 3 Pass new array to render function.

```
btnLaptops.addEventListener('click', () => {  
  
  // 1. Create filtered subset  
  const onlyLaptops = products.filter(  
    p => p.category === 'laptops'  
  );  
  
  // 2. Overwrite UI with subset  
  renderProducts(onlyLaptops);  
  
});
```